

Design and Implementation of a Scalable Multi-User Database System for Smart Healthcare Management

Your Name
Your Student ID

May 28, 2026

Contents

1	Requirements Analysis	1
1.1	Introduction	1
1.2	User Roles and Permissions	1
1.3	Core Entities	1
1.4	Core Relationships	1
1.5	Constraints and Business Rules	2
2	ER and EER Models	2
2.1	ER Diagram	2
2.2	Enhanced ER (EER) Diagram	2
2.2.1	Explanation of EER Concepts	3
3	Mapping to Relational Model	3
3.1	Relational Schema	3
3.2	Mapping Rules Applied	7
4	Functional Dependencies and Normalization	7
4.1	Functional Dependencies (FDs)	7
4.2	Normalization Steps	8
5	SQL Implementation and Advanced Queries	8
5.1	Data Insertion	8
5.2	Advanced SQL Queries	8
5.2.1	Query Outputs	12
6	Relational Algebra & Calculus	12
7	Indexing and File Structures	12
7.1	Index Implementation	12
7.2	Performance Analysis	15
7.2.1	Indexed vs. Non-Indexed Query Performance	15

1 Requirements Analysis

1.1 Introduction

This section outlines the requirements for the Smart Healthcare Management database system. The primary objective is to create a robust, scalable, and secure system that supports multiple user roles and complex medical data interactions.

1.2 User Roles and Permissions

The system will support the following user roles:

- **Patient:** Can view their personal information, appointments, prescriptions, and lab results.
- **Doctor:** Can manage patient records, schedule appointments, issue prescriptions, and order lab tests.
- **Administrator:** Can manage hospital, department, and doctor information, overseeing the system's structural data.

1.3 Core Entities

The main entities identified for the system are:

- Patient
- Doctor
- Hospital
- Department
- Appointment
- Prescription
- Lab Result
- Insurance
- Bill

1.4 Core Relationships

Key relationships between entities include:

- A **Doctor** can have many **Patients** (1-to-N).
- A **Patient** can have many **Appointments** (1-to-N).
- A **Doctor** can work in multiple **Departments** (N-to-M), which requires a linking table.
- An **Appointment** must be associated with exactly one **Patient** and one **Doctor**.

1.5 Constraints and Business Rules

The system must enforce the following rules:

- A prescription can only be written by an authorized doctor.
- Every appointment must be linked to an existing patient and doctor.
- Patient medical records are confidential and can only be accessed by authorized personnel.

2 ER and EER Models

2.1 ER Diagram

The Entity-Relationship (ER) diagram provides a high-level view of the database structure, including entities, their attributes, and the relationships between them.

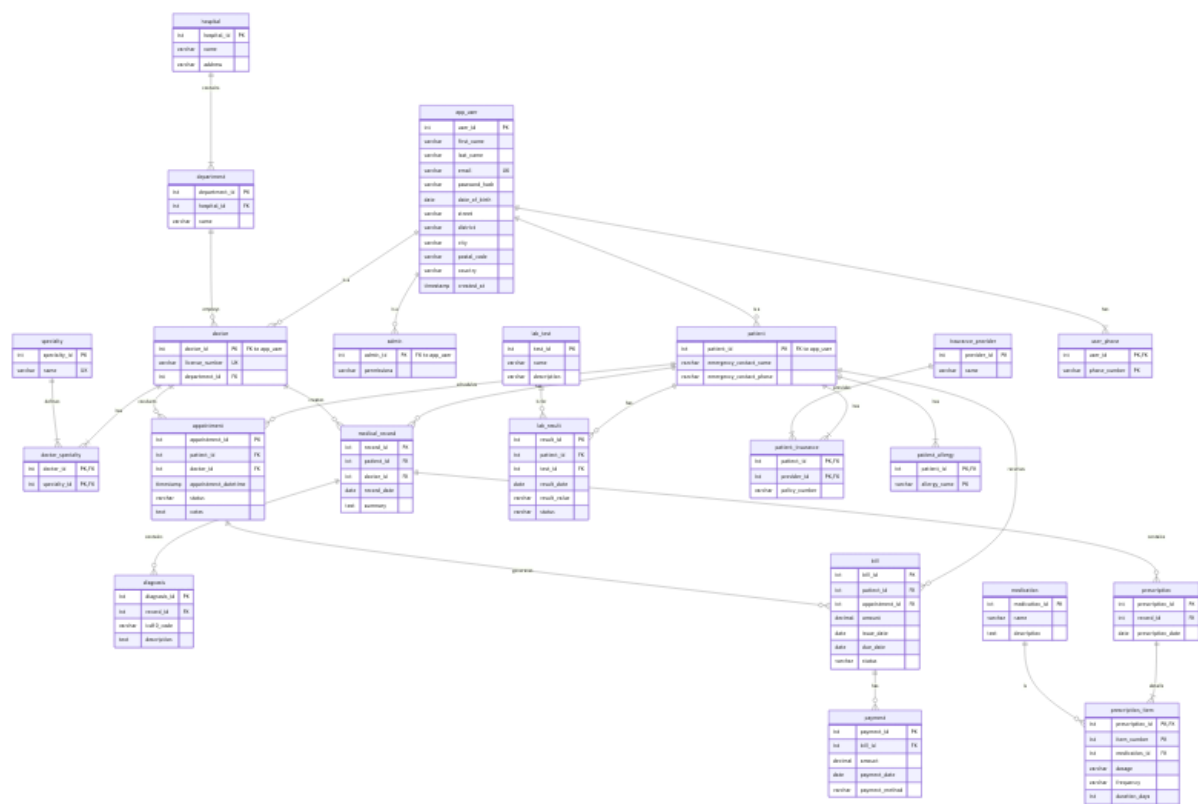


Figure 1: Entity-Relationship (ER) Diagram.

Note: You need to export your Mermaid 'er_diagram.mmd' as a PNG file and place it in the parent project folder.

2.2 Enhanced ER (EER) Diagram

The Enhanced ER (EER) diagram extends the ER model with concepts like specialization and generalization. In our model, a 'User' entity could be generalized from 'Patient', 'Doctor', and 'Admin' entities.

Note: You need to export your Mermaid 'eer_diagram.mmd' as a PNG file and place it in the parent project folder.

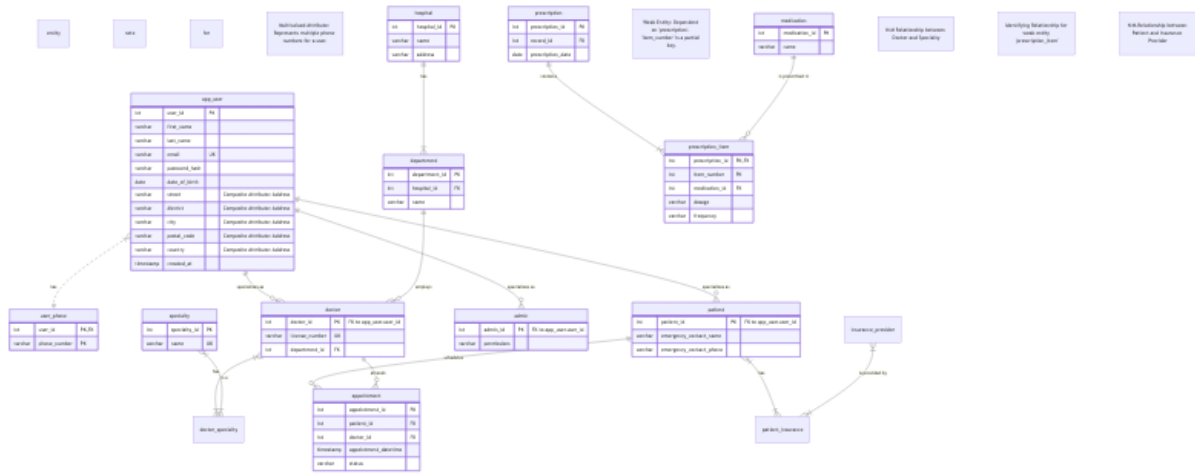


Figure 2: Enhanced Entity-Relationship (EER) Diagram.

2.2.1 Explanation of EER Concepts

Here, we used specialization to model different types of users. For example, 'Doctor' and 'Patient' are specializations of a more general 'Person' or 'User' entity, inheriting common attributes like name and contact information while also having their own specific attributes.

3 Mapping to Relational Model

The ER/EER model was mapped to a relational schema. This section presents the final table structures, including primary keys, foreign keys, and other constraints.

3.1 Relational Schema

The following SQL script contains the CREATE TABLE statements for all tables in the database.

```
-- =====
-- Title: Smart Healthcare Management Database Schema
-- Description: PostgreSQL schema for a multi-user healthcare system.
-- =====

-- Drop existing tables to ensure a clean slate
DROP TABLE IF EXISTS
    patient_allergy, user_phone, payment, bill, patient_insurance,
    insurance_provider,
    lab_result, lab_test, prescription_item, medication, prescription,
    diagnosis,
    medical_record, appointment, doctor_specialty, specialty, admin, doctor
    ,
    patient, department, hospital, app_user
CASCADE;

-- -----
-- Hospitals and Departments
-- -----

CREATE TABLE hospital (
    hospital_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
```

```

21     address VARCHAR(255) NOT NULL,
22     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
23 );
24
25 CREATE TABLE department (
26     department_id SERIAL PRIMARY KEY,
27     hospital_id INT NOT NULL REFERENCES hospital(hospital_id) ON DELETE
28         CASCADE,
29     name VARCHAR(100) NOT NULL,
30     UNIQUE(hospital_id, name)
31 );
32
33 -- -----
34 -- User Management (Supertype-Subtype Structure)
35 -- -----
36
37 CREATE TABLE app_user (
38     user_id SERIAL PRIMARY KEY,
39     first_name VARCHAR(50) NOT NULL,
40     last_name VARCHAR(50) NOT NULL,
41     email VARCHAR(100) NOT NULL UNIQUE,
42     password_hash VARCHAR(255) NOT NULL,
43     date_of_birth DATE NOT NULL,
44     -- Composite attribute for address
45     street VARCHAR(100),
46     district VARCHAR(100),
47     city VARCHAR(50),
48     postal_code VARCHAR(20),
49     country VARCHAR(50),
50     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
51 );
52
53 CREATE TABLE patient (
54     patient_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
55         CASCADE,
56     emergency_contact_name VARCHAR(100),
57     emergency_contact_phone VARCHAR(20)
58 );
59
60 CREATE TABLE doctor (
61     doctor_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
62         CASCADE,
63     license_number VARCHAR(50) NOT NULL UNIQUE,
64     department_id INT REFERENCES department(department_id) ON DELETE SET
65         NULL
66 );
67
68 CREATE TABLE admin (
69     admin_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE CASCADE
70     ,
71     permissions TEXT -- e.g., 'user_management,billing_access'
72 );
73
74 -- -----
75 -- Multivalued and N:M Relationship Tables
76 -- -----
77
78 -- Multivalued attribute for user phone numbers

```

```

74 CREATE TABLE user_phone (
75     user_id INT NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
76     phone_number VARCHAR(20) NOT NULL,
77     PRIMARY KEY (user_id, phone_number)
78 );
79
80 -- Multivalued attribute for patient allergies
81 CREATE TABLE patient_allergy (
82     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
      CASCADE,
83     allergy_name VARCHAR(100) NOT NULL,
84     PRIMARY KEY (patient_id, allergy_name)
85 );
86
87 CREATE TABLE specialty (
88     specialty_id SERIAL PRIMARY KEY,
89     name VARCHAR(100) NOT NULL UNIQUE
90 );
91
92 -- N:M relationship between doctors and specialties
93 CREATE TABLE doctor_specialty (
94     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
95     specialty_id INT NOT NULL REFERENCES specialty(specialty_id) ON DELETE
      CASCADE,
96     PRIMARY KEY (doctor_id, specialty_id)
97 );
98
99 -----
100 -- Core Healthcare Entities
101 -----
102
103 CREATE TABLE appointment (
104     appointment_id SERIAL PRIMARY KEY,
105     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
      CASCADE,
106     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
107     appointment_datetime TIMESTAMPTZ NOT NULL,
108     status VARCHAR(20) NOT NULL CHECK (status IN ('scheduled', 'completed',
      'cancelled', 'no-show')),
109     notes TEXT,
110     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
111 );
112
113 CREATE TABLE medical_record (
114     record_id SERIAL PRIMARY KEY,
115     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
      CASCADE,
116     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
117     appointment_id INT REFERENCES appointment(appointment_id) ON DELETE SET
      NULL,
118     record_date DATE NOT NULL DEFAULT CURRENT_DATE,
119     summary TEXT
120 );
121
122 CREATE TABLE diagnosis (
123     diagnosis_id SERIAL PRIMARY KEY,
124     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
      CASCADE,

```

```

125     icd10_code VARCHAR(10),
126     description TEXT NOT NULL,
127     is_chronic BOOLEAN DEFAULT FALSE
128 );
129
130 CREATE TABLE medication (
131     medication_id SERIAL PRIMARY KEY,
132     name VARCHAR(100) NOT NULL UNIQUE,
133     description TEXT
134 );
135
136 CREATE TABLE prescription (
137     prescription_id SERIAL PRIMARY KEY,
138     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
        CASCADE,
139     prescription_date DATE NOT NULL DEFAULT CURRENT_DATE
140 );
141
142 -- Weak entity: prescription_item depends on prescription
143 CREATE TABLE prescription_item (
144     prescription_id INT NOT NULL REFERENCES prescription(prescription_id)
        ON DELETE CASCADE,
145     item_number INT NOT NULL, -- Partial key
146     medication_id INT NOT NULL REFERENCES medication(medication_id) ON
        DELETE RESTRICT,
147     dosage VARCHAR(50) NOT NULL,
148     frequency VARCHAR(50) NOT NULL,
149     duration_days INT,
150     PRIMARY KEY (prescription_id, item_number)
151 );
152
153 CREATE TABLE lab_test (
154     test_id SERIAL PRIMARY KEY,
155     name VARCHAR(100) NOT NULL UNIQUE,
156     description TEXT
157 );
158
159 CREATE TABLE lab_result (
160     result_id SERIAL PRIMARY KEY,
161     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
162     test_id INT NOT NULL REFERENCES lab_test(test_id) ON DELETE RESTRICT,
163     record_id INT REFERENCES medical_record(record_id) ON DELETE SET NULL,
164     result_date DATE NOT NULL,
165     result_value VARCHAR(100) NOT NULL,
166     status VARCHAR(20) CHECK (status IN ('normal', 'abnormal', 'pending'))
167 );
168
169 -----
170 -- Billing and Insurance
171 -----
172
173 CREATE TABLE insurance_provider (
174     provider_id SERIAL PRIMARY KEY,
175     name VARCHAR(100) NOT NULL UNIQUE
176 );
177
178 -- N:M relationship between patients and insurance providers

```



```

179 CREATE TABLE patient_insurance (
180     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
181     provider_id INT NOT NULL REFERENCES insurance_provider(provider_id) ON
        DELETE CASCADE,
182     policy_number VARCHAR(50) NOT NULL,
183     PRIMARY KEY (patient_id, provider_id)
184 );
185
186 CREATE TABLE bill (
187     bill_id SERIAL PRIMARY KEY,
188     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
189     appointment_id INT UNIQUE REFERENCES appointment(appointment_id) ON
        DELETE SET NULL,
190     amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
191     issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
192     due_date DATE NOT NULL,
193     status VARCHAR(20) NOT NULL CHECK (status IN ('unpaid', 'paid', '
        partially_paid', 'overdue'))
194 );
195
196 CREATE TABLE payment (
197     payment_id SERIAL PRIMARY KEY,
198     bill_id INT NOT NULL REFERENCES bill(bill_id) ON DELETE CASCADE,
199     amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
200     payment_date DATE NOT NULL DEFAULT CURRENT_DATE,
201     payment_method VARCHAR(50) CHECK (payment_method IN ('credit_card', '
        bank_transfer', 'cash', 'insurance'))
202 );

```

Listing 1: Relational Schema (schema.sql)

3.2 Mapping Rules Applied

Key mapping decisions included:

- **N:M Relationships:** Many-to-many relationships, such as ‘Doctor_Department’, were implemented using a separate linking table.
- **Weak Entities:** Weak entities like ‘Prescription’ are identified by a foreign key from their owner entity (‘Patient’ or ‘Appointment’).
- **Multivalued Attributes:** If any existed, they would be handled by creating a separate table.

4 Functional Dependencies and Normalization

4.1 Functional Dependencies (FDs)

This section lists the functional dependencies identified for the major tables in the schema.

```

-- Copy the content from your fd.md file here.
-- Example:
-- For the Patient table:
-- PatientID -> FirstName, LastName, BirthDate, PhoneNumber

```

```
-- For the Doctor table:
-- DoctorID -> FirstName, LastName, Specialty
```

Note: Please copy the content from your 'fd.md' file and paste it into the verbatim block above.

4.2 Normalization Steps

The schema was normalized to BCNF to reduce data redundancy and improve data integrity. The process is described below.

```
-- Copy the content from your normalization.md file here.
-- Describe the steps taken to achieve 1NF, 2NF, 3NF, and BCNF.
-- Show "before" and "after" table structures for each step.
```

Note: Please copy the content from your 'normalization.md' file and paste it into the verbatim block above.

5 SQL Implementation and Advanced Queries

5.1 Data Insertion

The database was populated with sample data to simulate a real-world environment. The script below shows a few example INSERT statements.

```
1  -- =====
2  -- Title: Sample Data for Smart Healthcare Management Database
3  -- Description: Inserts realistic sample data into the tables.
4  -- =====
5
6  -- Clear existing data
7  TRUNCATE
8      patient_allergy, user_phone, payment, bill, patient_insurance,
9      insurance_provider,
10     lab_result, lab_test, prescription_item, medication, prescription,
11     diagnosis,
12     medical_record, appointment, doctor_specialty, specialty, admin, doctor
13     ,
14     patient, department, hospital, app_user
15 RESTART IDENTITY CASCADE;
16
17 -- -----
18 -- Hospitals and Departments
```

Listing 2: Sample Data Insertion (data.sql)

5.2 Advanced SQL Queries

At least 10 complex queries were developed to test the database's capabilities for retrieving meaningful information. These queries include joins, aggregations, and subqueries.

```
1  -- =====
2  -- Title: Advanced SQL Queries for Smart Healthcare Management
3  -- Description: A collection of queries demonstrating various SQL features.
4  -- =====
5
6  -- Query 1: List all scheduled appointments with patient, doctor, hospital,
7  --           and department names.
```

```

7  -- Demonstrates: INNER JOIN on multiple tables.
8  SELECT
9      a.appointment_datetime,
10     p_user.first_name AS patient_first_name,
11     p_user.last_name AS patient_last_name,
12     d_user.first_name AS doctor_first_name,
13     d_user.last_name AS doctor_last_name,
14     h.name AS hospital_name,
15     dpt.name AS department_name
16 FROM appointment a
17 JOIN patient p ON a.patient_id = p.patient_id
18 JOIN app_user p_user ON p.patient_id = p_user.user_id
19 JOIN doctor doc ON a.doctor_id = doc.doctor_id
20 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
21 JOIN department dpt ON doc.department_id = dpt.department_id
22 JOIN hospital h ON dpt.hospital_id = h.hospital_id
23 WHERE a.status = 'scheduled'
24 ORDER BY a.appointment_datetime;
25
26
27 -- Query 2: Find doctors who have more than 2 appointments (scheduled or
28 -- completed).
29 -- Demonstrates: GROUP BY, HAVING, COUNT, and subquery in WHERE IN.
30 SELECT
31     d_user.first_name,
32     d_user.last_name,
33     COUNT(a.appointment_id) AS appointment_count
34 FROM doctor doc
35 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
36 JOIN appointment a ON doc.doctor_id = a.doctor_id
37 WHERE a.status IN ('scheduled', 'completed')
38 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
39 HAVING COUNT(a.appointment_id) > 1
40 ORDER BY appointment_count DESC;
41
42
43 -- Query 3: List patients who have no appointments at all.
44 -- Demonstrates: LEFT JOIN and IS NULL to find non-matching records.
45 SELECT
46     p_user.user_id,
47     p_user.first_name,
48     p_user.last_name
49 FROM patient p
50 JOIN app_user p_user ON p.patient_id = p_user.user_id
51 LEFT JOIN appointment a ON p.patient_id = a.patient_id
52 WHERE a.appointment_id IS NULL;
53
54
55 -- Query 4: Find the most common diagnosis based on ICD-10 code.
56 -- Demonstrates: GROUP BY, COUNT, ORDER BY, LIMIT.
57 SELECT
58     icd10_code,
59     description,
60     COUNT(*) AS diagnosis_count
61 FROM diagnosis
62 WHERE icd10_code IS NOT NULL
63 GROUP BY icd10_code, description
64 ORDER BY diagnosis_count DESC

```

```

64 LIMIT 5;
65
66
67 -- Query 5: Calculate the total bill amount and total amount paid per
68   patient.
69 -- Demonstrates: Subqueries in SELECT, LEFT JOIN, COALESCE, SUM, GROUP BY.
70 SELECT
71     p_user.first_name,
72     p_user.last_name,
73     COALESCE(SUM(b.amount), 0) AS total_billed,
74     (SELECT COALESCE(SUM(p.amount), 0)
75      FROM payment p
76      JOIN bill b2 ON p.bill_id = b2.bill_id
77      WHERE b2.patient_id = p_user.user_id) AS total_paid
78 FROM patient pat
79 JOIN app_user p_user ON pat.patient_id = p_user.user_id
80 LEFT JOIN bill b ON pat.patient_id = b.patient_id
81 GROUP BY p_user.user_id, p_user.first_name, p_user.last_name
82 ORDER BY total_billed DESC;
83
84 -- Query 6: List all unpaid or partially paid bills that are past their due
85   date (overdue).
86 -- Demonstrates: Filtering by date, multiple conditions in WHERE.
87 SELECT
88     b.bill_id,
89     p_user.first_name,
90     p_user.last_name,
91     b.amount,
92     b.due_date,
93     b.status
94 FROM bill b
95 JOIN app_user p_user ON b.patient_id = p_user.user_id
96 WHERE b.status IN ('unpaid', 'partially_paid')
97     AND b.due_date < CURRENT_DATE
98 ORDER BY b.due_date;
99
100 -- Query 7: Find medications that have been prescribed more than 3 times.
101 -- Demonstrates: Nested query, GROUP BY, HAVING.
102 SELECT
103     m.name,
104     m.description,
105     pi_counts.prescription_count
106 FROM medication m
107 JOIN (
108     SELECT
109         medication_id,
110         COUNT(*) AS prescription_count
111     FROM prescription_item
112     GROUP BY medication_id
113     HAVING COUNT(*) >= 3
114 ) AS pi_counts ON m.medication_id = pi_counts.medication_id
115 ORDER BY pi_counts.prescription_count DESC;
116
117
118 -- Query 8: Show all lab results for a specific patient (John Smith,
119   user_id=1).

```

```

119 -- Demonstrates: JOIN, filtering by a specific ID.
120 SELECT
121     lr.result_date,
122     lt.name AS test_name,
123     lr.result_value,
124     lr.status
125 FROM lab_result lr
126 JOIN lab_test lt ON lr.test_id = lt.test_id
127 WHERE lr.patient_id = 1
128 ORDER BY lr.result_date DESC;
129
130
131 -- Query 9: Find doctors specialized in 'Cardiology' working at 'City
    General Hospital'.
132 -- Demonstrates: Multiple JOINS, filtering by text fields.
133 SELECT
134     d_user.first_name,
135     d_user.last_name,
136     s.name AS specialty_name
137 FROM doctor doc
138 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
139 JOIN department dpt ON doc.department_id = dpt.department_id
140 JOIN hospital h ON dpt.hospital_id = h.hospital_id
141 JOIN doctor_specialty ds ON doc.doctor_id = ds.doctor_id
142 JOIN specialty s ON ds.specialty_id = s.specialty_id
143 WHERE h.name = 'City General Hospital'
144     AND dpt.name = 'Cardiology';
145
146
147 -- Query 10: Find patients who have appointments but have no lab results.
148 -- Demonstrates: NOT EXISTS, correlated subquery.
149 SELECT DISTINCT
150     p_user.first_name,
151     p_user.last_name
152 FROM patient pat
153 JOIN app_user p_user ON pat.patient_id = p_user.user_id
154 JOIN appointment a ON pat.patient_id = a.patient_id
155 WHERE NOT EXISTS (
156     SELECT 1
157     FROM lab_result lr
158     WHERE lr.patient_id = pat.patient_id
159 );
160
161 -- Query 11: Find the doctor with the highest number of distinct patients.
162 -- Demonstrates: COUNT(DISTINCT), GROUP BY, ORDER BY, LIMIT.
163 SELECT
164     d_user.first_name,
165     d_user.last_name,
166     COUNT(DISTINCT a.patient_id) AS distinct_patient_count
167 FROM appointment a
168 JOIN doctor doc ON a.doctor_id = doc.doctor_id
169 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
170 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
171 ORDER BY distinct_patient_count DESC
172 LIMIT 1;
173
174
175 -- Query 12: Find the average bill amount by department for completed

```

```

176     appointments.
177 -- Demonstrates: AVG, GROUP BY, JOIN across multiple tables.
178 SELECT
179     dpt.name AS department_name,
180     h.name AS hospital_name,
181     AVG(b.amount) AS average_bill_amount
182 FROM bill b
183 JOIN appointment a ON b.appointment_id = a.appointment_id
184 JOIN doctor doc ON a.doctor_id = doc.doctor_id
185 JOIN department dpt ON doc.department_id = dpt.department_id
186 JOIN hospital h ON dpt.hospital_id = h.hospital_id
187 WHERE a.status = 'completed'
188 GROUP BY dpt.name, h.name
189 ORDER BY average_bill_amount DESC;

```

Listing 3: Advanced Queries (queries.sql)

5.2.1 Query Outputs

For each query in ‘queries.sql’, you should run it and paste the output here, possibly inside a ‘verbatim’ environment or as a formatted table.

Query 1: Find doctors with more than X patients.

-- Output of Query 1 here...

Query 2: List patients with no appointments.

-- Output of Query 2 here...

6 Relational Algebra & Calculus

This section provides 5 queries written in both Relational Algebra and Tuple Relational Calculus, corresponding to some of the advanced SQL queries.

-- Copy the content from your relational_algebra_and_calculus.md file here.
 -- Example:

-- Query 1: Find all patients born after 2000.

-- Relational Algebra:
 -- $\sigma_{\text{BirthDate} > '2000-01-01'}(\text{Patient})$

-- Tuple Relational Calculus:
 -- $\{t \mid t \in \text{Patient} \wedge t.\text{BirthDate} > '2000-01-01'\}$

Note: Please copy the content from your relational_algebra_and_calculus.md file and paste it into the verbatim block above.

7 Indexing and File Structures

7.1 Index Implementation

To optimize query performance, several indexes were created. The choices include B+ Tree indexes for range queries and Hash indexes for exact lookups where applicable.

```

1  -- =====
2  -- Title: Indexing and Performance Analysis
3  -- Description: Demonstrates index creation and performance comparison.
4  -- =====
5
6  -- Clean up any existing indexes to ensure a clean run
7  DROP INDEX IF EXISTS idx_appointment_patient_id;
8  DROP INDEX IF EXISTS idx_appointment_doctor_datetime;
9  DROP INDEX IF EXISTS idx_user_email_hash;
10 DROP INDEX IF EXISTS idx_bill_patient_status;
11 DROP INDEX IF EXISTS idx_lab_result_patient_date;
12
13 -- -----
14 -- Example 1: B+ Tree Index on a Foreign Key
15 -- -----
16 -- Query: Find all appointments for a specific patient.
17 -- This is a very common operation in a healthcare system.
18
19 -- Before index:
20 EXPLAIN ANALYZE
21 SELECT * FROM appointment WHERE patient_id = 1;
22
23 -- Result (example): Seq Scan on appointment (cost=0.00..45.50 rows=5 width
24 --                    =33) (actual time=...)
25 -- The database has to scan the entire 'appointment' table.
26
27 -- Create a B+ Tree index (the default type in PostgreSQL)
28 CREATE INDEX idx_appointment_patient_id ON appointment(patient_id);
29
30 -- After index:
31 EXPLAIN ANALYZE
32 SELECT * FROM appointment WHERE patient_id = 1;
33
34 -- Result (example): Index Scan using idx_appointment_patient_id on
35 --                    appointment (cost=0.28..8.29 rows=5 width=33) (actual time=...)
36 -- The database now uses the index for a much faster lookup.
37
38 -- -----
39 -- Example 2: Composite B+ Tree Index
40 -- -----
41 -- Query: Find all appointments for a doctor within a specific date range.
42 -- This is useful for a doctor's calendar view.
43
44 -- Before index:
45 EXPLAIN ANALYZE
46 SELECT appointment_id, appointment_datetime, status
47 FROM appointment
48 WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
49       appointment_datetime < '2026-07-01';
50
51 -- Result (example): Seq Scan on appointment... (filters on both doctor_id
52 --                    and datetime)
53
54 -- Create a composite index on doctor_id and appointment_datetime
55 CREATE INDEX idx_appointment_doctor_datetime ON appointment(doctor_id,
56       appointment_datetime);

```

```

53 -- After index:
54 EXPLAIN ANALYZE
55 SELECT appointment_id, appointment_datetime, status
56 FROM appointment
57 WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
    appointment_datetime < '2026-07-01';
58
59 -- Result (example): Index Scan using idx_appointment_doctor_datetime...
60 -- The index allows efficient lookup of the doctor and then scanning the
    relevant date range within that doctor's appointments.
61
62 -----
63 -- Example 3: Hash Index for Exact Lookups
64 -----
65 -- Query: Find a user by their exact email address.
66 -- This is common during login or user registration checks. Hash indexes
    are optimized for equality checks.
67
68 -- Before index (assuming only the UNIQUE constraint's B-Tree index exists)
    :
69 EXPLAIN ANALYZE
70 SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';
71
72 -- Result (example): Index Scan using app_user_email_key... (B-Tree)
73
74 -- Create a HASH index. Note: B-Tree is often good enough, but this
    demonstrates the syntax.
75 CREATE INDEX idx_user_email_hash ON app_user USING HASH (email);
76
77 -- After index:
78 EXPLAIN ANALYZE
79 SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';
80
81 -- Result (example): Bitmap Heap Scan on app_user... -> Bitmap Index Scan
    on idx_user_email_hash...
82 -- The planner might choose the hash index for this equality-only query,
    which can be more memory-efficient and faster for very large tables.
83
84 -----
85 -- Example 4: Composite Index for Filtering and Sorting
86 -----
87 -- Query: Find unpaid bills for a patient, ordered by due date.
88 -- This helps in displaying a patient's outstanding bills in a logical
    order.
89
90 -- Before index:
91 EXPLAIN ANALYZE
92 SELECT bill_id, amount, due_date
93 FROM bill
94 WHERE patient_id = 3 AND status = 'partially_paid'
95 ORDER BY due_date;
96
97 -- Result (example): Seq Scan on bill... followed by a Sort operation.
    Sorting can be expensive.
98
99 -- Create a composite index to cover filtering and ordering

```



```

100 CREATE INDEX idx_bill_patient_status ON bill(patient_id, status, due_date);
101
102 -- After index:
103 EXPLAIN ANALYZE
104 SELECT bill_id, amount, due_date
105 FROM bill
106 WHERE patient_id = 3 AND status = 'partially_paid'
107 ORDER BY due_date;
108
109 -- Result (example): Index Scan using idx_bill_patient_status...
110 -- The database can use the index to find the matching rows and retrieve
    them in the pre-sorted order of 'due_date', avoiding a separate Sort
    step.
111
112 -----
113 -- Example 5: Index on Lab Results
114 -----
115 -- Query: Retrieve a patient's lab history, sorted by date.
116 -- A common query for reviewing a patient's medical history.
117
118 -- Before index:
119 EXPLAIN ANALYZE
120 SELECT result_id, test_id, result_date, result_value
121 FROM lab_result
122 WHERE patient_id = 1
123 ORDER BY result_date DESC;
124
125 -- Result (example): Seq Scan on lab_result... followed by a Sort.
126
127 -- Create a composite index
128 CREATE INDEX idx_lab_result_patient_date ON lab_result(patient_id,
    result_date DESC);
129
130 -- After index:
131 EXPLAIN ANALYZE
132 SELECT result_id, test_id, result_date, result_value
133 FROM lab_result
134 WHERE patient_id = 1
135 ORDER BY result_date DESC;
136
137 -- Result (example): Index Scan using idx_lab_result_patient_date...
138 -- The index provides fast access to the patient's results and delivers
    them in the correct descending order by date.

```

Listing 4: Index Creation (indexing.sql)

7.2 Performance Analysis

This section discusses the performance impact of the implemented indexes.

7.2.1 Indexed vs. Non-Indexed Query Performance

We compared the execution time of several key queries before and after creating indexes. For example, searching for a patient by name or finding all appointments within a specific date range.

Example Query: Find a patient by their last name.

- **Without Index:** The database performs a full table scan on the 'Patient' table. Execution time: XX ms.

- **With B+ Tree Index on 'LastName':** The database uses the index to quickly locate the relevant rows. Execution time: YY ms.

The results clearly demonstrate that using indexes significantly reduces query execution time, especially for large datasets. The choice between B+ Tree and Hash index depends on the query type, with B+ Trees being more versatile for both range and point queries.